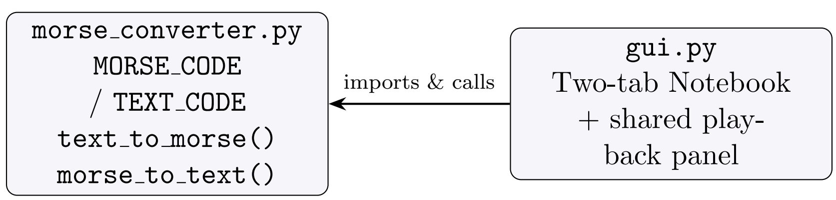
Morse Code Converter
Brief

Explainer document for the owner

1 What it is

A two-file Python project: `morse_converter.py` holds pure conversion logic (no UI), `gui.py` is a Tkinter (Python's built-in desktop GUI toolkit) window that wraps it in a live, two-way converter with a visual dot/dash playback animation.

2 Architecture at a glance



`gui.py` imports only the two conversion functions; it never touches the dictionaries directly. This separation means the conversion logic can be tested with no Tkinter dependency at all.

3 Conversion logic

`MORSE_CODE` maps every supported character (A–Z, 0–9, space, and common punctuation) to a string of dots and dashes; `TEXT_CODE` is the same mapping built in reverse with a one-line dict comprehension. The space character maps to `"/"`, the conventional Morse word separator.

`text_to_morse(text)` uppercases the input, looks up each character, and joins the patterns with single spaces (a literal input space becomes the three characters `" / "`). An unmapped character raises `ValueError` naming it, instead of a bare `KeyError`.

`morse_to_text(morse)` reverses this: it splits on the exact substring `" / "` to recover words, splits each word on whitespace to recover letters, looks each pattern up in `TEXT_CODE`, and raises `ValueError` on an unmapped pattern. Splitting on the exact three-character separator (rather than a bare `"/"`) is what keeps the round trip clean with no stray spaces.

Running `morse_converter.py` directly still gives the original one-line CLI prompt; `gui.py` is an additional interface, not a replacement.

4 The GUI

A `ttk.Notebook` (a themed tab bar) holds two tabs built from one shared helper function, differing only in which conversion function and labels they use: “Text to Morse” and “Morse to Text”. Each tab has an input `Text` box, an inline error label, a disabled (read-only) output `Text` box, and Copy/Clear buttons.

Every tab’s input box is bound to `<KeyRelease>`: on every keystroke, the handler reads the full input, calls the tab’s conversion function, and either fills the output box or shows the `ValueError` message in the error label. This runs even on invalid partial input (e.g. mid-typed Morse), which is exactly why the error is caught here rather than crashing the app.

Copy uses Tkinter’s built-in clipboard calls (no dependency); Clear resets both boxes and the error label for that tab.

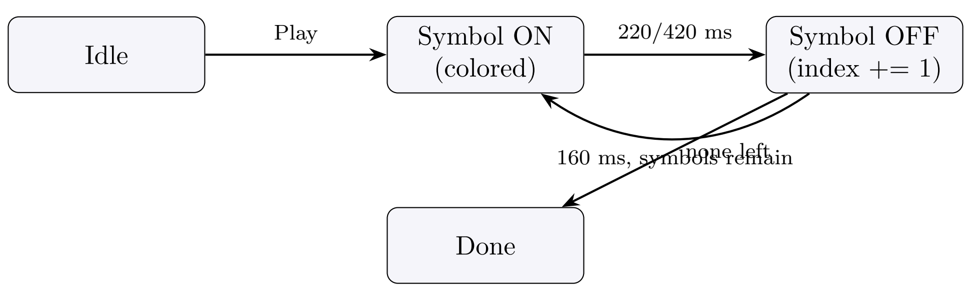
5 Playback panel

Below the tabs sits one shared `Canvas` and one Play button, used by both tabs rather than duplicated per tab. Pressing Play first decides *which* Morse string to animate: the Text-to-Morse tab’s output is checked first (it is tagged `"output"` as its Morse source); only if that is empty does it fall back to the Morse-to-Text tab’s input (tagged `"input"`, since that tab’s input box is the one holding Morse). This is a fixed priority order, not “whichever was edited last.”

The chosen string is filtered down to just its dot/dash characters, and one shape per symbol is drawn left to right: small ovals for dots, wider rectangles for dashes.

5.1 Timing state machine

Animation advances via `root.after(ms, callback)`, Tkinter’s non-blocking “run this once, later” scheduler — there is no separate thread. Each step schedules exactly one future step, so the chain is *recursive*, not a loop:



Dots stay lit 220 ms, dashes 420 ms (roughly double, matching the real Morse 1:3 dot-to-dash duration convention), with a 160 ms gap between symbols so consecutive marks read as distinct. Pressing Play again cancels any in-flight animation first, so repeated presses cannot stack.

6 Deliberately out of scope

No persistence, no network calls, no configuration file, no audio (playback is visual only — there is no cross-platform, dependency-free way to play a tone from the Python standard library), and no automated unit tests in the repository.

7 Glossary

- **Tkinter / ttk** — Python's built-in GUI toolkit; `ttk` is its themed widget set, used here for tabs, buttons, and frames.
- **Dict comprehension** — a compact Python expression that builds a dictionary from a loop in one line.
- `ValueError` — a built-in exception raised here to signal an invalid character or Morse symbol, instead of a raw `KeyError`.
- `root.after(ms, fn)` — schedules `fn` to run once after `ms` milliseconds without blocking the GUI's event loop.